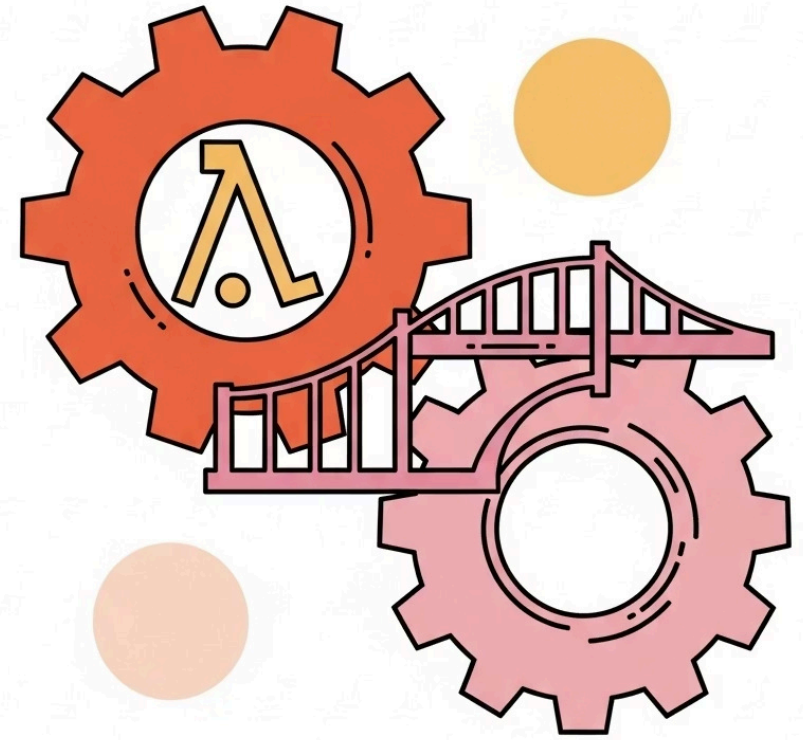


Interfaces Funcionales en Java

La base de las expresiones lambda y Streams

Una guía clara para entender el fundamento de la programación funcional en Java

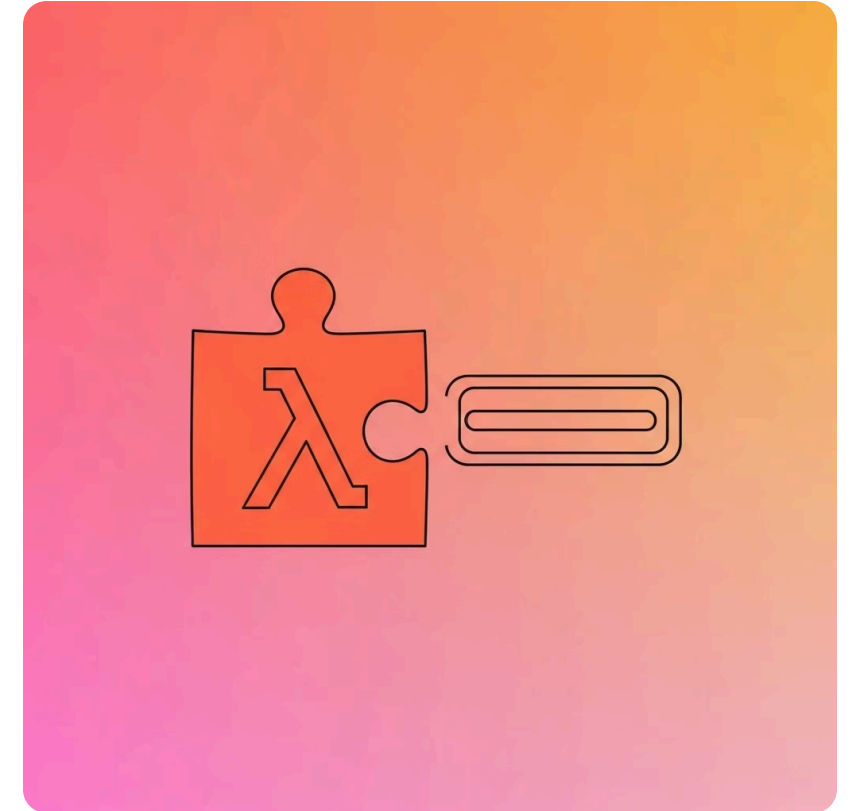


¿Por qué son importantes?

Las expresiones lambda son poderosas, pero necesitan **un lugar donde encajar** dentro del sistema de tipos de Java.

Ese lugar es el **método abstracto** de una interfaz funcional.

Sin interfaces funcionales, las lambdas simplemente **no funcionarían** en Java.



Lambda

Expresión anónima concisa



Interfaz Funcional

Contrato con un único método



Método Abstracto

Implementado por la lambda

Definición y Características

Una interfaz funcional tiene:

☐ **Un único método abstracto**

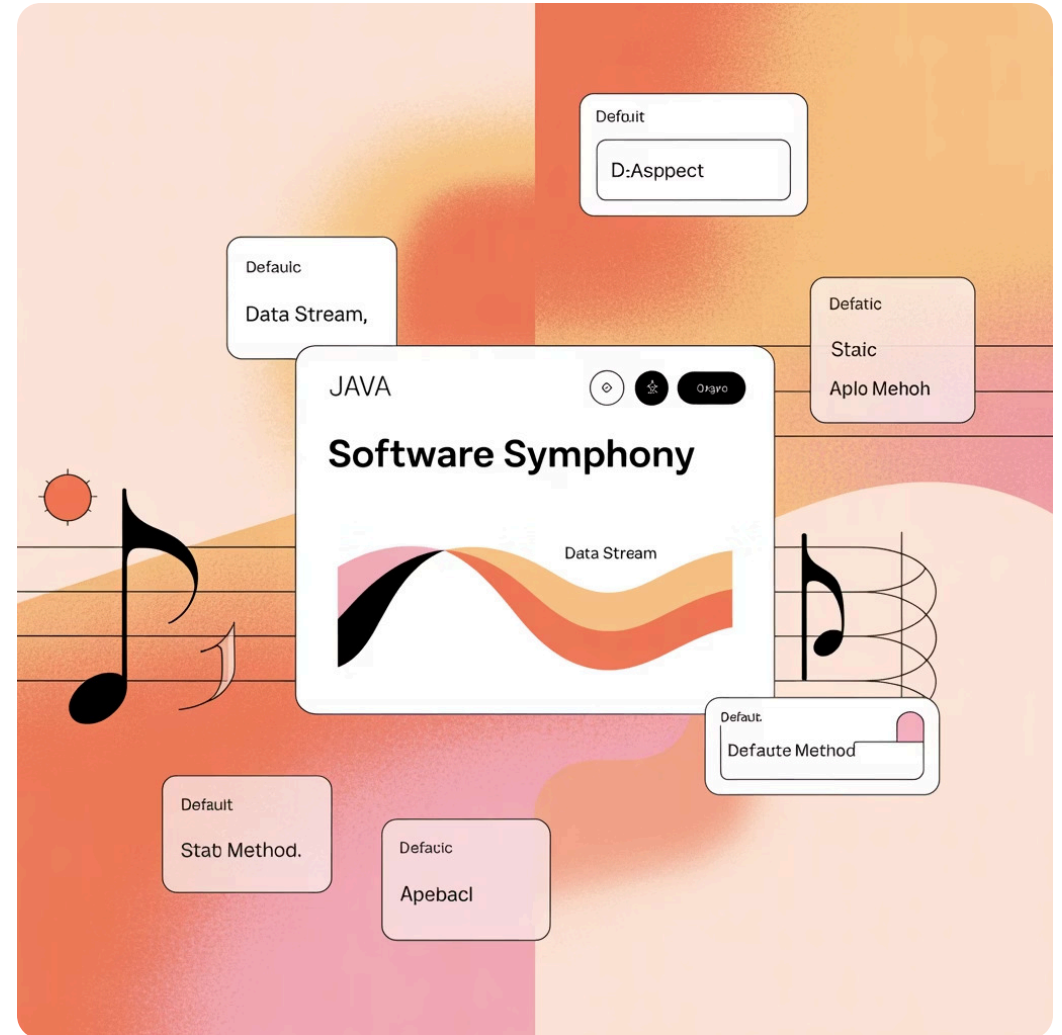
Este es el requisito fundamental. No puede tener más ni menos de un método abstracto.

☐ **Puede tener métodos default y static**

Estos métodos adicionales no afectan su condición de interfaz funcional.

☐ **Anotación @FunctionalInterface**

Es opcional pero recomendada. El compilador verificará que cumpla los requisitos.



La clave: exactamente un método abstracto

Ejemplo propio: Adder

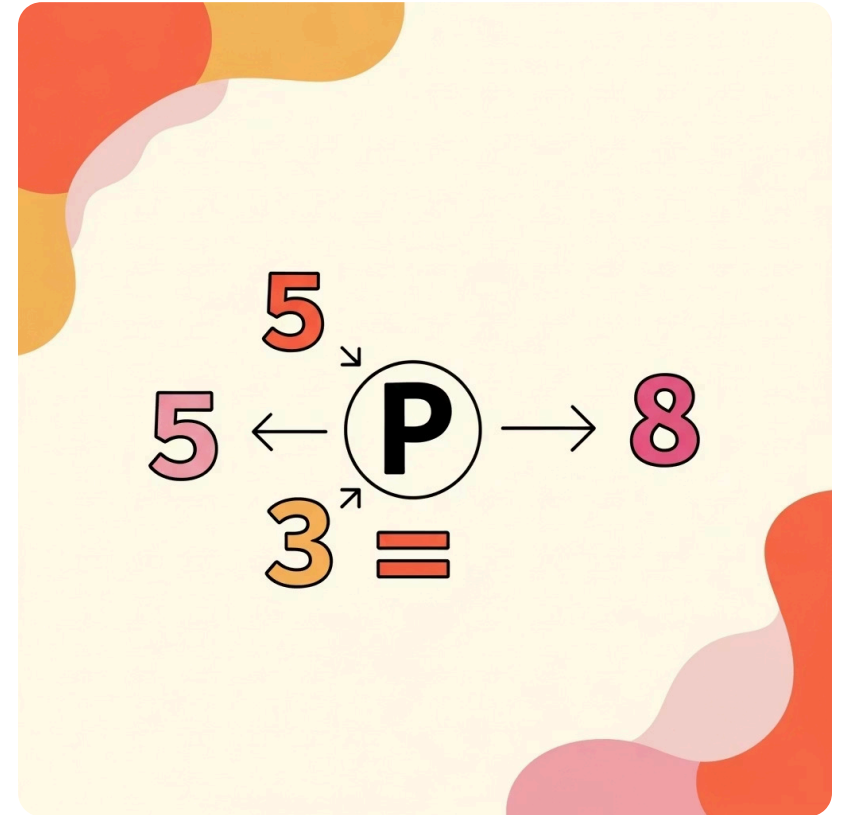
```
@FunctionalInterface
public interface Adder {
    int add(int a, int b);
}

// Implementación con lambda
Adder adder = (a, b) -> a + b;

// Uso
System.out.println(adder.add(5, 3)); // 8
```

La interfaz `Adder` define **un único método abstracto** que recibe dos enteros y devuelve su suma.

La expresión lambda `(a, b) -> a + b` implementa ese método de forma concisa.



Más ejemplos propios

Podemos crear nuestras propias interfaces funcionales para operaciones específicas:



Subtractor

```
@FunctionalInterface
interface Subtractor {
    int subtract(int a, int b);
}
```

Implementación: `(a, b) -> a - b`



Multiplier

```
@FunctionalInterface
interface Multiplier {
    int multiply(int a, int b);
}
```

Implementación: `(a, b) -> a * b`



Divider

```
@FunctionalInterface
interface Divider {
    double divide(int a, int b);
}
```

Implementación: `(a, b) -> (double)a / b`

Estas interfaces son simples pero ilustran perfectamente el propósito y uso de las interfaces funcionales con expresiones lambda.

Interfaces funcionales de Java

Java proporciona interfaces funcionales predefinidas en el paquete `java.util.function`:



Predicate<T>

```
boolean test(T t)
```

Evalúa una condición sobre un objeto y devuelve **true/false**.

Ejemplo: `Predicate<String> isLong = s -> s.length() > 5;`



Function<T,R>

```
R apply(T t)
```

Transforma un objeto de tipo T en otro de tipo R.

Ejemplo: `Function<String,Integer> length = s -> s.length();`



Consumer<T>

```
void accept(T t)
```

Consume un objeto pero no devuelve nada (void).

Ejemplo: `Consumer<String> print = s -> System.out.println(s);`

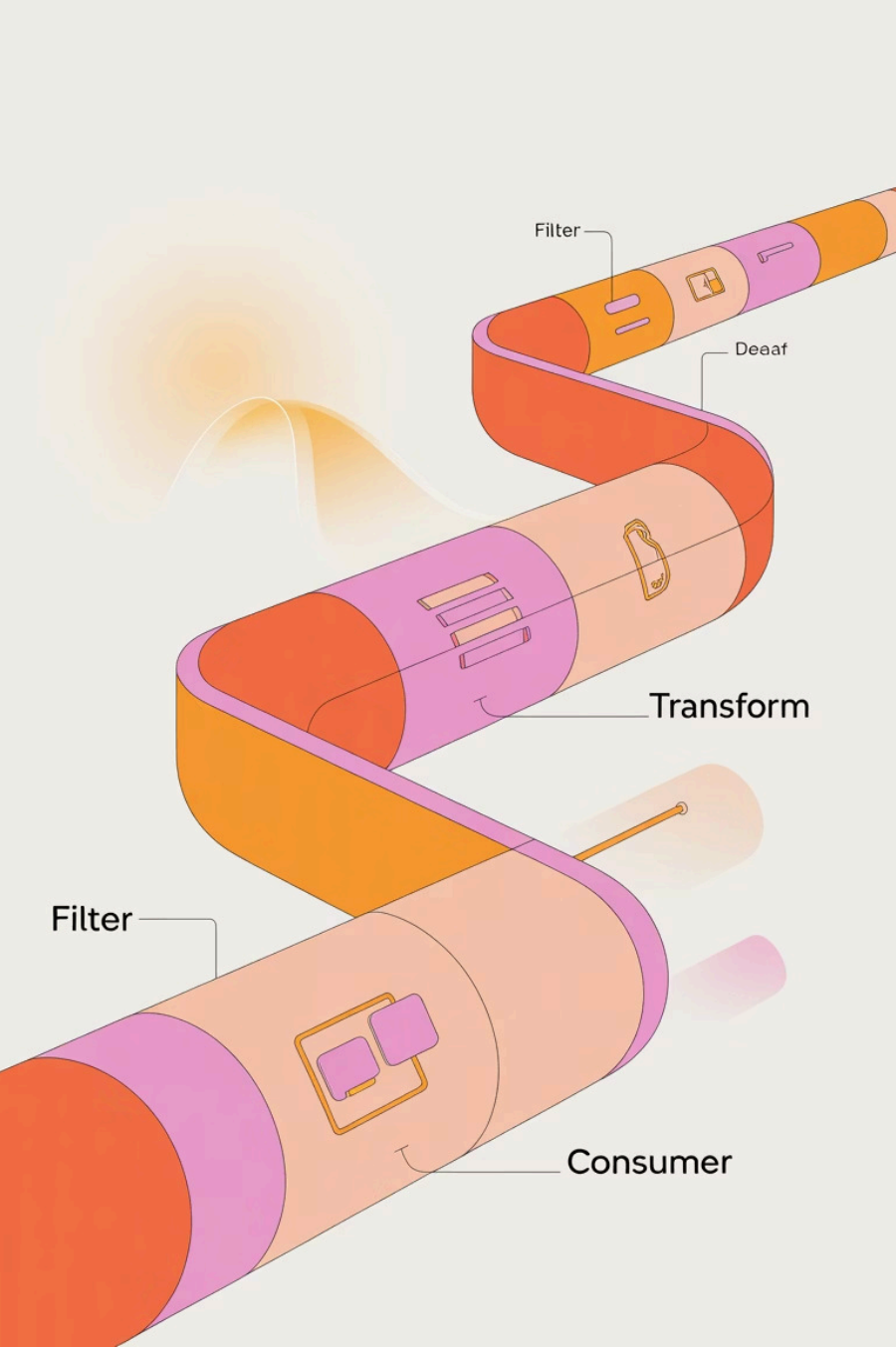


Supplier<T>

```
T get()
```

Produce un objeto sin recibir ningún argumento.

Ejemplo: `Supplier<Double> random = () -> Math.random();`



Conexión con Streams

Las interfaces funcionales son fundamentales para el API de Streams en Java:

```
List<String> nombres = List.of("Ana", "Pedro", "Juan");  
nombres.stream()  
    .filter(n -> n.length() > 3) // Predicate  
    .map(String::toUpperCase)    // Function  
    .forEach(System.out::println); // Consumer
```



filter()

Usa `Predicate<T>` para filtrar elementos



map()

Usa `Function<T,R>` para transformar elementos

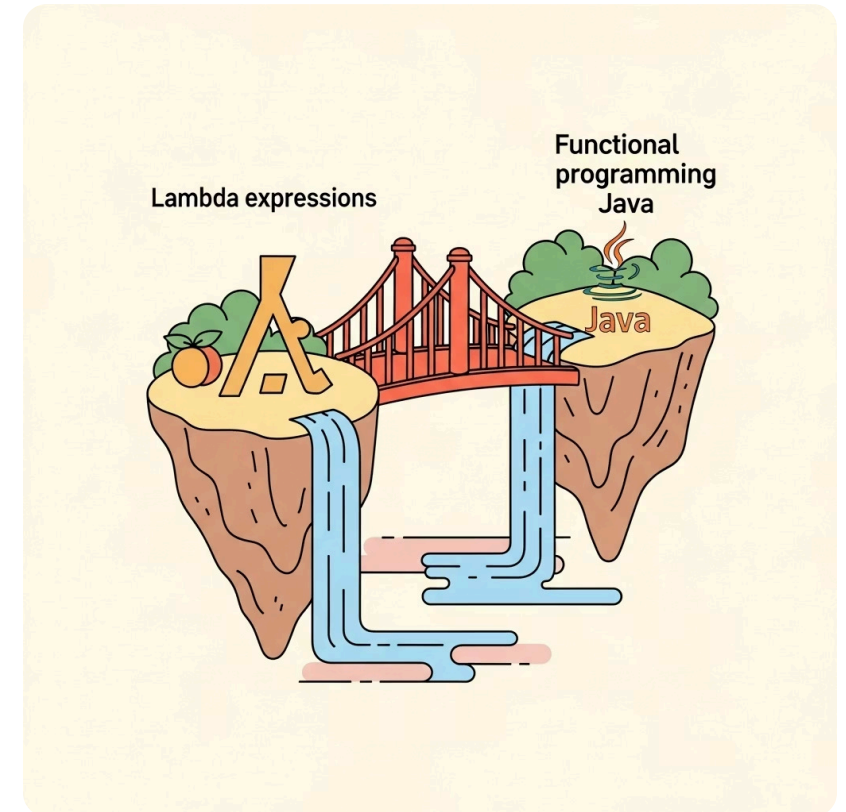


forEach()

Usa `Consumer<T>` para procesar elementos

Resumen

- ☐ Una interfaz funcional = un método abstracto
Este es el principio fundamental que define a las interfaces funcionales.
- ☐ Las lambdas implementan ese método
Proporcionando un código conciso y legible para operaciones funcionales.
- ☐ Podemos crear nuestras propias interfaces
O utilizar las predefinidas en `java.util.function`.
- ☐ Streams dependen de ellas para trabajar
Forman la **base que conecta lambdas con programación funcional** en Java.



Las interfaces funcionales son el **puente fundamental** que permitió incorporar programación funcional a Java, un lenguaje orientado a objetos.